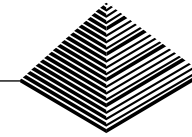
26



Architecture in the Cloud

There was a time when every household, town, farm or village had its own water well. Today, shared public utilities give us access to clean water by simply turning on the tap; cloud computing works in a similar fashion.
—Vivek Kundra

If you have read anything about the history of computing, you will have read about time-sharing. This was the era, in the late 1960s and the 1970s, sandwiched between eras when individuals had sole, although limited, access to multimillion-dollar computers and when individuals had access to their own personal computers. Time-sharing involved multiple users (maybe as many as several hundred) simultaneously accessing a powerful mainframe computer through a terminal, potentially remote from the mainframe. The operating system on the mainframe made it appear as if each user had sole access to that computer except, possibly, for performance considerations. The driving force behind the development of time-sharing was economic; it was infeasible to provide every user with a multimillion-dollar computer, but efficiently sharing this expensive but powerful resource was the solution.

In some ways, cloud computing is a re-creation of that era. In fact, some of the basic techniques—such as virtualization—that are used in the cloud today date from that period. Any user of an application in the cloud does not need to know that the application and the data it uses are situated several time zones away, and that thousands of other users are sharing it. Of course, with the advent of the Internet, the availability of much more powerful computers today, and the requirement for controlled sharing, designing the architecture for a cloud-based application is much different from designing the architecture for a time-sharing-based application. The driving forces, however, remain much the same. The

economics of using the cloud as a deployment platform are so compelling that few organizations today can afford to ignore this set of technologies.

In this chapter we introduce cloud concepts, and we discuss various service models and deployment options for the cloud, the economic justification for the cloud, the base architectures and mechanisms that make the cloud work, and some sample technologies. We will conclude by discussing how an architect should approach building a system in the cloud.

26.1 Basic Cloud Definitions

The essential characteristics of cloud computing (based, in part, on definitions provided by the U.S. National Institute of Standards and Technology, or NIST) are the following:

1. *On-demand self-service.* A resource consumer can unilaterally provision computing services, such as server time and network storage, as needed automatically without requiring human interaction with each service's provider. This is sometimes called empowerment of end users of computing resources. Examples of resources include storage, processing, memory, network bandwidth, and virtual machines.
2. *Ubiquitous network access.* Cloud services and resources are available over the network and accessed through standard networking mechanisms that promote use by a heterogeneous collection of clients. For example, you can effectively run large applications on small platforms such as smart phones, laptops, and tablets by running the resource-intensive portion of those applications on the cloud. This capability is independent of location and device; all you need is a client and the Internet.
3. *Resource pooling.* The cloud provider's computing resources are pooled. In this way they can efficiently serve multiple consumers. The provider can dynamically assign physical and virtual resources to consumers, according to their instantaneous demands.
4. *Location independence.* The location independence provided by ubiquitous network access is generally a good thing. It does, however, have one potential drawback. The consumer generally has less control over, or knowledge of, the location of the provided resources than in a traditional implementation. This can have drawbacks for data latency. The consumer may be able to ameliorate this drawback by specifying abstract location information (e.g., country, state, or data center).
5. *Rapid elasticity.* Due to resource pooling, it is easy for capabilities to be rapidly and elastically provisioned, in some cases automatically, to quickly scale out or in. To the consumer, the capabilities available for provisioning often appear to be virtually unlimited.

6. *Measured service.* Cloud systems automatically control and optimize resource use by leveraging a metering capability for the chosen service (e.g., storage, processing, bandwidth, and user accounts). Resource usage can be monitored, controlled, and reported so that consumers of the services are billed only for what they use.
7. *Multi-tenancy.* Multi-tenancy is the use of a single application that is responsible for supporting distinct classes or users. Each class or user has its own set of data and access rights, and different users or classes of users are kept distinct by the application.

26.2 Service Models and Deployment Options

In this section we discuss more terminology and basic concepts. First we discuss the most important models for a consumer using the cloud.

Cloud Service Models

Software as a Service (SaaS). The consumer in this case is an end user. The consumer uses applications that happen to be running on a cloud. The applications can be as varied as email, calendars, video streaming, and real-time collaboration. The consumer does not manage or control the underlying cloud infrastructure, including network, servers, operating systems, storage, or even individual application capabilities, with the possible exception of limited user-specific application configuration settings.

Platform as a Service (PaaS). The consumer in this case is a developer or system administrator. The platform provides a variety of services that the consumer may choose to use. These services can include various database options, load-balancing options, availability options, and development environments. The consumer deploys applications onto the cloud infrastructure using programming languages and tools supported by the provider. The consumer does not manage or control the underlying cloud infrastructure, including network, servers, operating systems, or storage, but has control over the deployed applications and possibly application hosting environment configurations. Some levels of quality attributes (e.g., uptime, response time, security, fault correction time) may be specified by service-level agreements (SLAs).

Infrastructure as a Service (IaaS). The consumer in this case is a developer or system administrator. The capability provided to the consumer is to provision processing, storage, networks, and other fundamental computing resources where the consumer is able to deploy and run arbitrary software, which

can include operating systems and applications. The consumer can, for example, choose to create an instance of a virtual computer and provision it with some specific version of Linux. The consumer does not manage or control the underlying cloud infrastructure but has control over operating systems, storage, deployed applications, and possibly limited control of select networking components (e.g., host firewalls). Again, SLAs are often used to specify key quality attributes.

Deployment Models

The various deployment models for the cloud are differentiated by who owns and operates the cloud. It is possible that a cloud is owned by one party and operated by a different party, but we will ignore that distinction and assume that the owner of the cloud also operates the cloud.

There are two basic models and then two additional variants of these. The two basic models are private cloud and public cloud:

- *Private cloud.* The cloud infrastructure is owned solely by a single organization and operated solely for applications owned by that organization. The primary purpose of the organization is not the selling of cloud services.
- *Public cloud.* The cloud infrastructure is made available to the general public or a large industry group and is owned by an organization selling cloud services.

The two variants are community cloud and hybrid cloud:

- *Community cloud.* The cloud infrastructure is shared by several organizations and supports a specific community that has shared concerns (e.g., mission, security requirements, policy, and compliance considerations).
- *Hybrid cloud.* The cloud infrastructure is a composition of two or more clouds (private, community, or public) that remain unique entities. The consumer will deploy applications onto some combination of the constituent cloud. An example is an organization that utilizes a private cloud except for periods when spikes in load lead to servicing some requests from a public cloud. Such a technique is called “cloud bursting.”

26.3 Economic Justification

In this section we discuss three economic distinctions between (cloud) data centers based on their size and the technology that they use:

1. Economies of scale

2. Utilization of equipment
3. Multi-tenancy

The aggregated savings of the three items we discuss may be as large as 80 percent for a 100,000-server data center compared to a 10,000-server data center. Economic considerations have made almost all startups deploy into the cloud. Many larger enterprises deploy a portion of their applications into the cloud, and almost every enterprise with substantial computation needs at least considers the cloud as a deployment platform.

Economies of Scale

Large data centers are inherently less expensive to operate per unit measure, such as cost per gigabyte, than smaller data centers. Large data centers may have hundreds of thousands of servers. Smaller data centers have servers numbered in the thousands or maybe even the hundreds. The cost of maintaining a data center depends on four factors:

1. *Cost of power.* The cost of electricity to operate a data center currently is 15 to 20 percent of the total cost of operation. The per-server power usage tends to be significantly lower in large data centers than in smaller ones because of the ability to share items such as racks and switches. In addition, large power users can negotiate significant discounts (as much as 50 percent) compared to the retail rates that operators of small data centers must pay. Some areas of the United States provide power at significantly lower rates than the national average, and large data centers can be located in those areas. Finally, organizations such as Google are buying or building innovative and cheaper power sources, such as on- and offshore wind farms and rooftop solar energy.
2. *Infrastructure labor costs.* Large data centers can afford to automate many of the repetitive management tasks that are performed manually in smaller data centers. In a traditional data center, an administrator can service approximately 140 servers, whereas in a cloud data center, the same administrator can service thousands of servers.
3. *Security and reliability.* Maintaining a given level of security, redundancy, and disaster recovery essentially requires a fixed level of investment. Larger data centers can amortize that investment over their larger number of servers and, consequently, the cost per server will be lower.
4. *Hardware costs.* Operators of large data centers can get discounts on hardware purchases of up to 30 percent over smaller buyers.

These economies of scale depend only on the size of the data center and do not depend on the deployment model being used. Operators of public clouds

have priced their offerings so that many of the cost savings are passed on to their consumers.

Utilization of Equipment

Common practice in nonvirtualized data centers is to run one application per server. This is caused by the dependency of many enterprise applications on particular operating systems or even particular versions of these operating systems. One result of the restriction of one application per server is extremely low utilization of the servers. Figures of 10 to 15 percent utilization for servers are quoted by several different vendors.

Use of virtualization technology, described in Section 26.4, allows for easy co-location of distinct applications and their associated operating systems on the same server hardware. The effect of this co-location is to increase the utilization of servers. Furthermore, variations in workload can be managed to further increase the utilization. We look at five different sources of variation and discuss how they might affect the utilization of servers:

1. *Random access.* End users may access applications randomly. For example, the checking of email is for some people continuous and for others time-boxed into a particular time period. The more users that can be supported on a single server, the more likely that the randomness of their accesses will end up imposing a uniform load on the server.
2. *Time of day.* Those services that are workplace related, unsurprisingly, tend to be more heavily used during the work day. Those that are consumer related tend to be heavily used during evening hours. Co-locating different services with different time-of-day usage patterns will increase the overall utilization of a server. Furthermore, time differences among geographically distinct locations will also affect utilization patterns and can be considered when planning deployment schedules.
3. *Time of year.* Some applications respond to dates as well as time of day. Consumer sites will see increases during the Christmas shopping season, and floral sites will see increases around Valentine's Day and Mother's Day. Tax preparation software will see increases around the tax return submission due date. Again, these variations in utilization are predictable and can be considered when planning deployment schedules.
4. *Resource usage patterns.* Not all applications use resources in the same fashion. Search, for example, is heavier in its usage of CPU than email but lighter in its use of storage. Co-locating applications with complementary resource usage patterns will increase the overall utilization of resources.
5. *Uncertainty.* Organizations must maintain sufficient capacity to support spikes in usage. Such spikes can be caused by news events if your site is a news provider, by marketing events if your site is consumer-facing, or even

sporting events because viewers of sporting events may turn to their computers during breaks in the action. Startups can face surges in demand if their product catches on more quickly than they can build capacity.

The first four sources of variation are supported by virtualization without reference to the cloud or the cloud deployment model. The last source of variation (uncertainty) depends on having a deployment model that can accommodate spikes in demand. This is the rationale behind cloud bursting, or keeping applications in a private data center and offloading spikes in demand to the public cloud. Presumably, a public cloud provider can deploy sufficient capacity to accommodate any single organization's spikes in demand.

Multi-tenancy

Multi-tenancy applications such as Salesforce.com or Microsoft Office 365 are architected explicitly to have a single application that supports distinct sets of users. The economic benefit of multi-tenancy is based on the reduction in costs for application update and management. Consider what is involved in updating an application for which each user has an individual copy on their own desktop. New versions must be tested by the IT department and then pushed to the individual desktops. Different users may be updated at different times because of disconnected operation, user resistance to updates, or scheduling difficulties. Incidents result because the new version may have some incompatibilities with older versions, the new version may have a different user interface, or users with old versions are unable to share information with users of the new version.

With a multi-tenant application, all of these problems are pushed from IT to the vendor, and some of them even disappear. Any update is available at the same instant to all of the users, so there are no problems with sharing. Any user interface changes are referred to the vendor's hotline rather than the IT hotline, and the vendor is responsible for avoiding incompatibilities for older versions.

The problems of upgrading do not disappear, but they are amortized over all of the users of the application rather than being absorbed by the IT department of every organization that uses the application. This amortization over more users results in a net reduction in the costs associated with installing an upgraded version of an application.

26.4 Base Mechanisms

In this section we discuss the base mechanisms that clouds use to provide their low-level services. In an IaaS instance, the cloud provides to the consumer a virtual machine loaded with a machine image. Virtualization is not a new concept;

it has been around since the 1960s. But today virtualization is economically enticing. Modern hardware is designed to support virtualization, and the overhead it adds has been measured to be just 1 percent per instance running on the bare hardware.

We will discuss the architecture of an IaaS platform in Section 26.5. In this section, we describe the concepts behind a virtual machine: the hypervisor and how it manages virtual machines, a storage system, and the network.

Hypervisor

A hypervisor is the operating system used to create and manage virtual machines. Because each virtual machine has its own operating system, a consumer application is actually managed by two layers of operating system: the hypervisor and the virtual machine operating system. The hypervisor manages the virtual machine operating system and the virtual machine operating system manages the consumer application. The key services used by the hypervisor to support the virtual machines it manages are a virtual page mapper and a scheduler. A hypervisor, of course, provides additional services and has a much richer structure than we present here, but these key services are the two that we will discuss.

Page Mapper

We begin by describing how virtual memory works on a bare (nonvirtualized) machine. All modern servers utilize virtual memory. Virtual memory allows an application to assume it has a large amount of memory in which to execute. The assumed memory is mapped into a much smaller physical memory through the use of page tables. The consumer application is divided into pages that are either in physical memory or temporarily residing on a disk. The page table contains the mapping of logical address (consumer application address) to physical address (actual machine address) or disk location. Figure 26.1 shows the consumer application executing its next instruction. This causes the CPU to generate a target address from which to fetch the next instruction or data item. The target address is used to address into a page table. The page table provides a physical address within the computer where the actual instruction or data item can be found if it is currently in main memory. If the physical address is not currently resident in the main memory of the computer, an interrupt is generated that causes a page that contains the target address to be loaded. This is the mechanism that allows a large (virtual) address space to be supported on much smaller physical memory.

Turning the virtual memory mechanism into a virtualization mechanism involves adding another level of indirection. Figure 26.2 shows a logical sequence that maps from the consumer application to a physical machine address. Modern processors contain many optimizations to make this process more efficient. A consumer application generates the next instruction with its target address. This target address is within the virtual machine in which the consumer application is

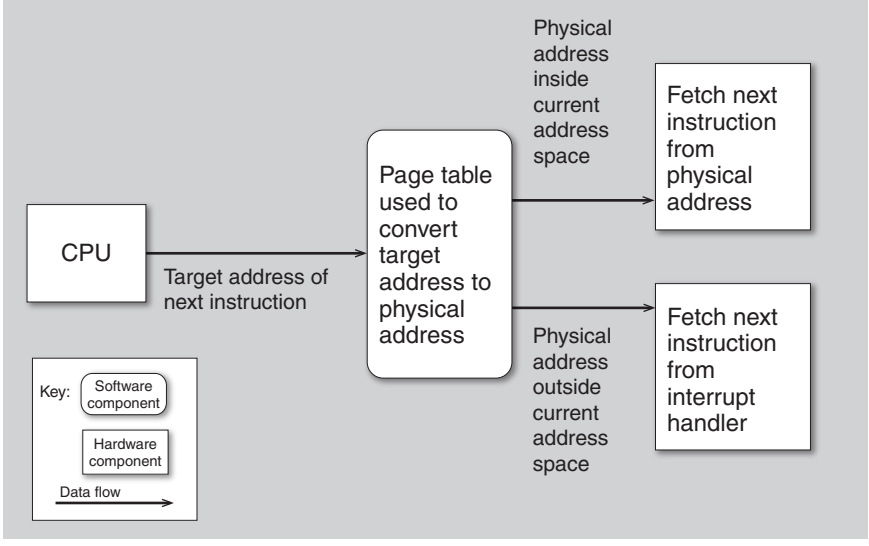


FIGURE 26.1 Virtual memory page table

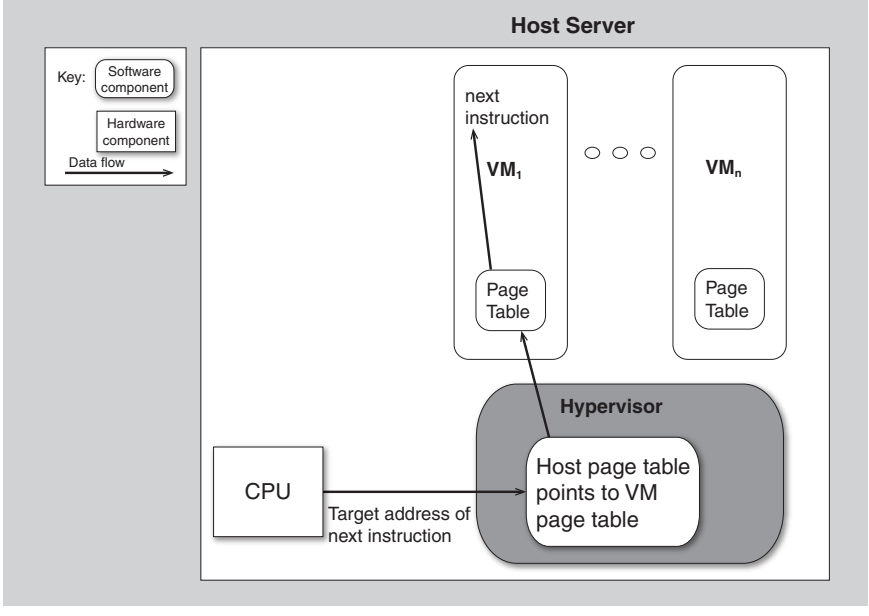


FIGURE 26.2 Adding a second level of indirection to determine which virtual machine the address references

executing. The virtual machine page table maps this target address to an address within the virtual machine based on the target address as before (or indicates that the page is not currently in memory). The address within the virtual machine is converted to a physical address by use of a page table within the hypervisor that manages the current virtual machines.

Scheduler

The hypervisor scheduler operates like any operating system scheduler. Whenever the hypervisor gets control, it decides on the virtual machine to which it will pass control. A simple round-robin scheduling algorithm assigns the processor to each virtual machine in turn, but many other possible scheduling algorithms exist. Choosing the correct scheduling algorithm requires you to make assumptions about the demand characteristics of the different virtual machines hosted within a single server. One area of research is the application of real-time scheduling algorithms to hypervisors. Real-time schedulers would be appropriate for the use of virtualization within embedded systems, but not necessarily within the cloud.

Storage

A virtual machine has access to a storage system for persistent data. The storage system is managed across multiple physical servers and, potentially, across clusters of servers. In this section we describe one such storage system: the Hadoop Distributed File System (HDFS).

We describe the redundancy mechanism used in HDFS as an example of the types of mechanisms used in cloud virtual file systems. HDFS is engineered for scalability, high performance, and high availability.

A component-and-connector view of HDFS within a cluster is shown in Figure 26.3. There is one NameNode process for the whole cluster, multiple DataNodes, and potentially multiple client applications. To explain the function of HDFS, we trace through a use case. We describe the successful use case for "write." HDFS also has facilities to handle failure, but we do not describe these. See the "For Further Reading" section for a reference to the HDFS failure-handling mechanisms.

For the "write" use case, we will assume that the file has already been opened. HDFS does not use locking to allow for simultaneous writing by different processes. Instead, it assumes a single writer that writes until the file is complete, after which multiple readers can read the file simultaneously. The application process has two portions: the application code and a client library specific to HDFS. The application code can write to the client using a standard (but overloaded) Java I/O call. The client buffers the information until a block of 64 MB has been collected. Two of the techniques used by HDFS for enhancing performance are the avoidance of locks and the use of 64-MB blocks as the only block

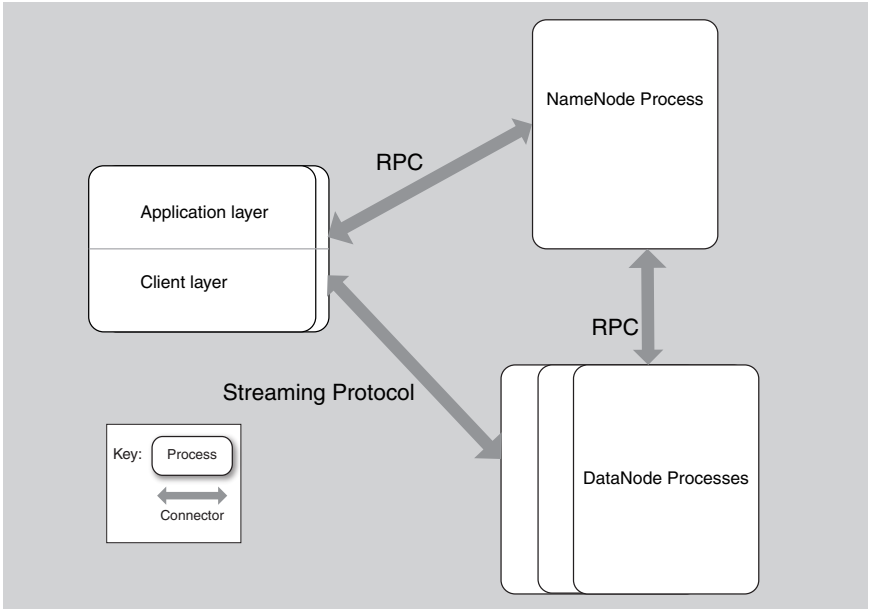


FIGURE 26.3 A component-and-connector view of an HDFS deployment. Each process exists on a distinct computer.

size supported. No substructure of the blocks is supported by HDFS. The blocks are undifferentiated byte strings. Any substructure and typing of the information is managed solely by the application. This is one example of a phenomenon that we will notice in portions of the cloud: moving application-specific functionality *up* the stack as opposed to moving it down the stack to the infrastructure.

For reliability purposes each block is replicated a parameterizable number of times, with a default of three. For each block to be written, the NameNode allocates DataNodes to write the replicas. The DataNodes are chosen based on two criteria: (1) their location—replicas are spread across racks to protect against the possibility that a rack fails; and (2) the dynamic load on the DataNode. Lightly loaded DataNodes are given preference over heavily loaded DataNodes to reduce the possibility of contention for the DataNodes among different files being simultaneously accessed.

Once the client has collected a buffer of 64 MB, it asks the NameNode for the identities of the DataNodes that will contain the actual replicas. The NameNode manages only metadata; it is not involved in the actual transfer or recording of data. These DataNode identities are sent from the NameNode to the client, which then treats them as a pipeline. At this point the client streams the block to the first DataNode in the pipeline. The first DataNode then streams the data to the second

DataNode in the pipeline, and so forth until the pipeline (of three DataNodes, unless the client has specified a different replication value) is completed. Each DataNode reports back to the client when it has successfully written the block, and also reports to the NameNode that it has successfully written the block.

Network

In this section we describe the basic concepts behind Internet Protocol (IP) addressing and how a message arrives at your computer. In Section 26.5 we discuss how an IaaS system manages IP addresses.

An IP address is assigned to every "device" on a network whether this device is a computer, a printer, or a virtual machine. The IP address is used both to identify the device and provide instructions on how to find it with a message. An IPv4 address is a constrained 32-bit number that is, typically, represented as four groups for human readability. For example, 192.0.2.235 is a valid IP address. The familiar names that we use for URLs, such as "http://www.pearsonhighered.com/", go through a translation process, typically through a domain name server (DNS), that results in a numeric IP address. A message destined for that IP address goes through a routing process to arrive at the appropriate location.

Every IP message consists of a header plus a payload. The header contains the source IP address and the destination IP address. IPv6 replaces the 32-bit number with a 128-bit number, but the header of an IP message still includes the source and destination IP addresses.

It is possible to replace the header of an IP message for various reasons. One reason is that an organization uses a gateway to manage traffic between external computers and computers within the organization. An IP address is either "public," meaning that it is unique within the Internet, or "private," meaning that multiple copies of the IP address are used, with each copy owned by a different organization. Private IP addresses must be accessed through a gateway into the organization that owns it. For outgoing messages, the gateway records the address of the internal machine and its target and replaces the source address in the TCP header with its own public IP address. On receipt of a return message, the gateway would determine the internal address for the message and overwrite the destination address in the header and then send the message onto the internal network. Network address translation (NAT) is the name of this process of translation.

26.5 Sample Technologies

Building on the base mechanisms, we now discuss some of the technologies that exist in the cloud. We begin by discussing the design of a generic IaaS platform,

then we move up the stack to a PaaS, and finally we discuss database technology in the cloud.

Infrastructure as a Service

Fundamentally, an IaaS installation provides three services: virtualized computation, virtualized networking, and a virtualized file system. In the previous section on base mechanisms, we described how the operating system for an individual server manages memory to isolate each virtual machine and how TCP/IP messages could be manipulated. An IaaS provides a management structure around these base concepts. That is, virtual machines must be allocated and deallocated, messages must be routed to the correct instance, and persistence of storage must be ensured.

We now discuss the architecture of a generic IaaS platform. Various providers will offer somewhat different services within different architectures. OpenStack is an open source movement to standardize IaaS services and interfaces, but as of this writing, it is still immature.

Figure 26.4 shows an allocation view of a generic cloud platform. Each server shown provides a different function to the platform, as we discuss next.

An IaaS installation has a variety of clusters. Each cluster may have thousands of physical servers. Each cluster has a *cluster manager* responsible for that cluster's

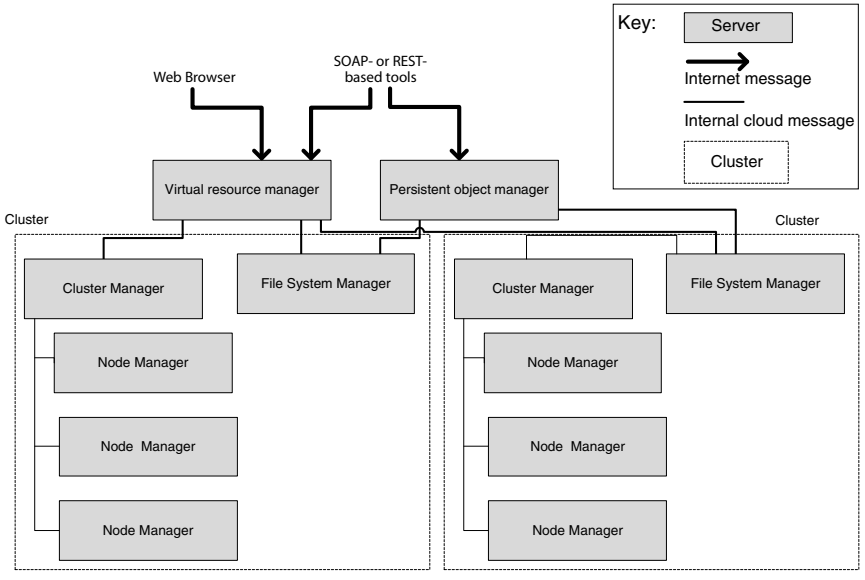


FIGURE 26.4 A generic cloud allocation view

resources. The *persistent object manager* supports the manipulation of persistent objects, and the *virtual resource managers* are in charge of the other virtualized resources. For requests for new resources, the virtual resource manager is in charge of determining which cluster manager will service the request. For requests sent to existing resources, the virtual resource manager is responsible for seeing that the requests get forwarded to the correct server. The virtual resource manager, in this case, acts as a gateway, as described in Section 26.4.

Some of the services that IaaS providers offer to support applications are these:

- *Automatic reallocation of IP addresses in the case of a failure of the underlying virtual machine instance.* This service is useful in case the instance has a public IP address. Unless the provider offers this service, the client must register the IP address of a replacement instance with a domain name server to ensure that messages are sent to the correct location.
- *Automatic scaling.* One of the virtues of the cloud is that new instances can be created or deleted relatively quickly in the event of a variation in demand. Detecting the variation in demand, allocating (or deleting) an instance in the event of a variation, and ensuring that the remaining instances are allocated their fair share of messages is another service that could be provided by the IaaS.

The persistent object manager is responsible for maintaining files that are intended to persist past the deletion of a virtual machine instance. It may maintain these files across multiple clusters in a variety of different geographic locations.

Failure of the underlying hardware is a common occurrence in a large data center, consequently the virtual resource manager has mechanisms to manage requests in the event of failure. These mechanisms are typically designed to maintain the availability of the IaaS infrastructure and do not extend to the applications deployed with the virtual machines. What this means in practice is that if you make a request for a new resource, it will be honored. If you make a request to an existing virtual machine instance, the infrastructure will guarantee that, if your virtual machine instance is active, your request is delivered. If, however, the host on which your virtual machine instance has been allocated has failed, then your virtual machine instance is no longer active and it is your responsibility as an application architect to install mechanisms to recognize a failure of your virtual machine instances and recover from them.

The *file system manager* manages the file system for each cluster. It is similar to the Hadoop Distributed File System that we discussed in Section 26.4. It also assumes that failure is a common occurrence and has mechanisms to replicate the blocks and to manage handoffs in the event of failures.

The cluster manager controls the execution of virtual machines running on the nodes within its clusters and manages the virtual networking between virtual machines and between virtual machines and external users.

The final piece of Figure 26.4 is the *node manager*; it (through the functionality of a hypervisor) controls virtual machine activities, including the execution, inspection, and termination of virtual machine instances.

A client initially requests a virtual machine instance and the virtual resource manager decides on which cluster the virtual machine instance should reside. It passes the instance request to the cluster manager, which in turn decides which node should host the virtual machine instance.

Subsequent requests are routed through the pieces of the generic infrastructure to the correct instance. The instance can create files using the file system manager. These files will either be deleted when the virtual machine instance is finished or will be persisted past the existence of the virtual machine instance. The choice is the client's as to how long storage is persisted. If the storage is persisted, it can be accessed independently of the creating instance through the persistence manager.

Platform as a Service

A Platform as a Service provides a developer with an integrated stack within the cloud to develop and deploy applications. IaaS provides virtual machines, and it is the responsibility of the developer using IaaS to provision the virtual machines with the software they desire. PaaS is preprovisioned with a collection of integrated software.

Consider a virtual machine provisioned with the LAMP (Linux, Apache, MySQL, PHP/Perl/Python) stack. The developer writes code in Python, for example, and has available the services provided by the other elements of the stack. Take this example and add automatic scaling across virtual machines based on customer load, automatic failure detection and recovery, backup/restore, security, operating system patch installation, and built-in persistence mechanisms. This yields a simple example of a PaaS.

The vendors offering PaaS and the substance of their offerings are rapidly evolving. Google and Microsoft are two of the current vendors.

1. The Google App Engine provides the developer with a development environment for Python or Java. Google manages deploying and executing developed code. Google provides a database service that is automatically replicated across data centers.
2. Microsoft Azure provides an operating system and development platform to access/develop applications on Microsoft data centers. Azure provides a development environment for applications running on Windows using .NET. It also provides for the automatic scaling and replication of instances. For example, if an application instance fails, then the Azure infrastructure will detect the failure and deploy another instance automatically. Azure also has a database facility that automatically keeps replicas of your databases.

Databases

A number of different forces have converged in the past decade, resulting in the creation of database systems that are substantially different from the relational database management systems (RDBMSs) that were prevalent during the 1980s and '90s.

- Massive amounts of data began to be collected from web systems. A search engine must index billions of pages. Facebook, today, has over 800 million users. Much of this data is processed sequentially and, consequently, the sophisticated indexing and query optimizations of RDBMSs are not necessary.
- Large databases are continually being created during various types of processing of web data. The creation and maintenance of databases using a traditional RDBMS requires a sophisticated data administrator.
- A theoretical result (the so-called CAP theorem) shows that it is not possible to simultaneously achieve consistency, availability, and partitioning. One of these properties must be sacrificed. For many applications, the choice is to sacrifice consistency and provide immediate availability and "eventual consistency." What this means, in practice, is that occasionally a user will access stale data, but updates will be subsequently available. The alternative approach, taken by RDBMSs, is to lock values and not allow access until they become consistent.
- The relational model is not the best model for some applications. The relational model assumes there is one data item for each row-value/column-name pair. One method for handling web searches, for example, is to store different versions of a single web page indexed by the same row-value/column-name pair so that the different versions of the web page can be quickly accessed and differences easily determined. Using the relational model requires that the system perform joins to retrieve all of the attributes associated with a particular row value. Joins are expensive from a performance perspective, and consequently, newly emerging database systems tend to not support joins and require storing data in a denormalized form.

These forces resulted in the creation of new types of databases with different data models and different access mechanisms. These new types of databases go under the name of NoSQL—although as Michael Stonebraker has pointed out, the existence or nonexistence of SQL within the database system is irrelevant to the rationale for their existence.

We discuss two open source NoSQL database systems: a key-value one (HBase) and a document-centric one (MongoDB) .

HBase

HBase is a key-value database system based on the BigTable database system developed by Google. Google uses BigTable to store data for many of their applications. The number of data items in a HBase database can be in the billions or trillions.

HBase supports tables, although there is no schema used. One column is designated as the key. The other columns are treated as field names. A data value is indexed by a row value, a column name, and a time stamp. Each row value/column name can contain multiple versions of the same data differentiated by time stamps.

One use of HBase is for web crawling. In this application, the row value is the URL for the web page. Each column name refers to an attribute of a web page that will support the analysis of the web page. For example, "contents" might be one column name. In the relational model, each row value/column name would retrieve the contents of the web page. Web pages change over time, however, and so in the relational model, there would need to be a separate column with the time stamp, and the primary key for the table would be the URL/time stamp. In HBase, the versions of the web page are stored together and retrieved by the URL value/"contents". All of the versions of the web page are retrieved, and it is the responsibility of the application to separate the versions of the web page and determine which one is desired based on the time stamp.

MongoDB

MongoDB uses a document-centric data model. You can think of it as storing objects rather than tables. An object contains all of the information associated with a particular concept without regard to whether relations among data items are stored in multiple different objects. Two distinct objects may have no field names in common, some field names in common, or all of the field names in common.

You may store links rather than data items. Links support the concept of joining different objects without requiring the maintenance of indices and query optimization. It is the responsibility of the application to follow the link.

Documents are stored in binary JavaScript Object Notation (JSON) form. Indices can be created on fields and can be used for retrieval, but there is no concept of primary versus secondary keys. A field is either indexed or it is not. Because the same field can occur in multiple different documents, a field is indexed wherever it occurs.

What Is Left Out of NoSQL Databases

One motivation for NoSQL databases is performance when accessing millions or billions of data items. To this end, several standard RDBMS facilities are omitted in NoSQL databases. If an application wishes to have these features, it must implement them itself. Mainly, the features are omitted for performance reasons.

- *Schemas.* NoSQL databases typically do not require schemas for their data model and, consequently, there is no checking of field names for consistency.
- *Transactions.* NoSQL typically does not support transactions. Transactions lock data items, which hinders performance. Applications use techniques

such as time stamps to determine whether fields have been modified through simultaneous access.

- *Consistency.* NoSQL databases are "eventually consistent." This means that after some time has passed, different replicas of a data item will have the same value, but in the interim, it is possible to run two successive queries that access the same data item and retrieve two different values.
- *Normalization.* NoSQL databases do not support joins. Joins are a requirement if you are to normalize your database.

26.6 Architecting in a Cloud Environment

Now we take the point of view of an architect who is designing a system to execute in the cloud. In some ways, the cloud is a platform, and architecting a system to execute in the cloud, especially using IaaS, is no different than architecting for any other distributed platform. That is, the architect needs to pay attention to usability, modifiability, interoperability, and testability, just as he or she would for any other platform. The quality attributes that have some significant differences are security, performance, and availability.

Security

Security, as always, has both technical and nontechnical aspects. The nontechnical aspects of security are items such as what trust is placed in the cloud provider, what physical security does the cloud provider utilize, how are employees of the cloud provider screened, and so forth. We will focus on the technical aspects of security.

Applications in the cloud are accessed over the Internet using standard Internet protocols. The security and privacy issues deriving from the use of the Internet are substantial but no different from the security issues faced by applications not hosted in the cloud. The one significant security element introduced by the cloud is multi-tenancy. Multi-tenancy means that your application is utilizing a virtual machine on a physical computer that is hosting multiple virtual machines. If one of the other tenants on your machine is malicious, what damage can they do to you?

There are four possible forms of attack utilizing multi-tenancy:

1. *Inadvertent information sharing.* Each tenant is given a set of virtual resources. Each virtual resource is mapped to some physical resource. It is possible that information remaining on a physical resource from one tenant may "leak" to another tenant.
2. *A virtual machine "escape."* A virtual machine is isolated from other virtual machines through the use of a distinct address space. It is possible,

however, that an attacker can exploit software errors in the hypervisor to access information they are not entitled to. Thus far, such attacks are extremely rare.

3. *Side-channel attacks.* It is possible for a malicious attacker to deduce information about keys and other sensitive information by monitoring the timing activity of the cache. Again, so far, this is primarily an academic exercise.
4. *Denial-of-service attacks.* Other tenants may use sufficient resources on the host computer so that your application is not able to provide service.

Some providers allow customers to reserve entire machines for their exclusive use. Although this defeats some of the economic benefits of using the cloud, it is a mechanism to prevent multi-tenancy attacks. An organization should consider possible attacks when deciding which applications to host in the cloud, just as they should when considering any hosting option.

Performance

The instantaneous computational capacity of any virtual machine will vary depending on what else is executing on that machine. Any application will need to monitor itself to determine what resources it is receiving versus what it will need.

One virtue of the cloud is that it provides an elastic host. Elasticity means that additional resources can be acquired as needed. An additional virtual machine, for example, will provide additional computational capacity. Some cloud providers will automatically allocate additional resources as needed, whereas other providers view requesting additional resources as the customer's responsibility.

Regardless of whether the provider automatically allocates additional resources, the application should be self-aware of both its current resource usage and its projected resource usage. The best the provider can do is to use general algorithms to determine whether there is a need to allocate or free resources. An application should have a better model of its own behavior and be better equipped to do its own allocation or freeing of resources. In the worst case, the application can compare its predictions to those of the provider to gain insight into what will happen. It takes time for the additional resources to be allocated and freed. The freeing of resources may not be instantaneously reflected in the charging algorithm used by the provider, and that charging algorithm also needs to be considered when allocating or freeing resources.

Availability

The cloud is assumed to be always available. But everything can fail. A virtual machine, for example, is hosted on a physical machine that can fail. The virtual

network is less likely to fail, but it too is fallible. It behooves the architect of a system to plan for failure.

The service-level agreement that Amazon provides for its EC2 cloud service provides a 99.95 percent guarantee of service. There are two ways of looking at that number: (1) That is a high number. You as an architect do not need to worry about failure. (2) That number indicates that the service may be unavailable for .05 percent of the time. You as an architect need to plan for that .05 percent.

Netflix is a company that streams videos to home television sets, and its reliability is an important business asset. Netflix also hosts much of its operation on Amazon EC2. On April 21, 2011, Amazon EC2 suffered a four-day sporadic outage. Netflix customers, however, were unaware of any problem.

Some of the things that Netflix did to promote availability that served them well during that period were reported in their tech blog. We discussed their Simian Army in Chapter 10. Some of the other things they did were applications of availability tactics that we discussed in Chapter 5.

- *Stateless services.* Netflix services are designed such that any service instance can serve any request in a timely fashion, so if a server fails, requests can be routed to another service instance. This is an application of the spare tactic, because the other service instance acts as a spare.
- *Data stored across zones.* Amazon provides what they call "availability zones," which are distinct data centers. Netflix ensured that there were multiple redundant hot copies of the data spread across zones. Failures were retried in another zone, or a hot standby was invoked. This is an example of the active redundancy tactic.
- *Graceful degradation.* The general principles for dealing with failure are applications of the degradation or the removal from service tactic:
  - Fail fast: Set aggressive timeouts such that failing components don't make the entire system crawl to a halt.
  - Fallbacks: Each feature is designed to degrade or fall back to a lower quality representation.
  - Feature removal: If a feature is noncritical, then if it is slow it may be removed from any given page.

The CAP Theorem

The CAP theorem—created by Eric Brewer at UC Berkeley—emerged over a decade ago. Unlike most theories postulated by academics, this one did not sink into obscurity but rather has grown in renown and influence since then. The theory states that there are three important properties of a distributed system managing shared data. These are the following:

- Consistency (C): the data will be consistent throughout the distributed system.
- Availability (A): the data will be highly available.
- Partitioning (P): the system will tolerate network partitioning.

And the theory further states that no system can achieve all of these properties simultaneously; the best we can hope for is to satisfy two out of three while sacrificing (to some extent) the third property. Brewer explains it thus:

> *The easiest way to understand CAP is to think of two nodes on opposite sides of a partition. Allowing at least one node to update state will cause the nodes to become inconsistent, thus forfeiting C. Likewise, if the choice is to preserve consistency, one side of the partition must act as if it is unavailable, thus forfeiting A. Only when nodes communicate is it possible to preserve both consistency and availability, thereby forfeiting P.*

In fact, there is really another important facet to the CAP theorem that has come to dominate the engineering challenge: latency. It wasn't part of the original acronym (although CLAP is certainly catchy), but a concern for latency now infuses much of the discussion of the tradeoffs in implementing NoSQL databases.

Creators of large-scale distributed NoSQL databases are constantly faced with tradeoffs. These days no one believes that you simply choose two of the three properties of CAP; the decisions are far richer and more subtle than that. For example, designers of these systems like to speak of "eventual consistency"—partitions are allowed to become inconsistent on a regular basis, but with bounds that are carefully engineered and monitored. They might want to specify that no more than *x* percent of the data should be stale at any given time, and it should not take more than *y* seconds to restore consistency (on average, or in the worst case). Another common tradeoff seen in practice is that availability and latency are typically favored over consistency. That is, a Facebook user should get quick response from the system, even if their newsfeed is slightly stale.

All of this adds complexity to the system. The designer has to choose between faster/less consistent and slower/more consistent (as well as a host of other quality issues). And the mechanisms for achieving eventual consistency—caching, replication, message retries, timeouts, and so forth—are themselves nontrivial. Consistency, partitioning, latency, and availability are four qualities that can be traded off with NoSQL databases. In addition, other quality attributes—interoperability, security, and so forth—also add complexity, and so the tradeoffs involved can get more and more complicated.

Alas, this is, increasingly, the world that we live in. Systems with global reach and enormous bases of distributed data are not going away anytime soon. So as architects we need to be prepared to deal with tradeoffs and complexities for the foreseeable future.

—RK

26.7 Summary

The cloud has become a viable alternative for the hosting of data centers primarily for economic reasons. It provides an elastic set of resources through the use of virtual machines, virtual networks, and virtual file systems.

The cloud can be used to provide infrastructure, platforms, or services. Each of these has its own characteristics.

NoSQL database systems arose in reaction to the overhead introduced by large relational database management systems. NoSQL database systems frequently use a data model based on key-value or documents and do not provide support for common database services such as transactions.

Architecting in the cloud means that the architect should pay attention to specific aspects of quality attributes that are substantially different in cloud environments, namely: performance, availability, and security.

26.8 For Further Reading

Dealers of Lightning: Xerox PARC and the Dawn of the Computer Age [Hiltzik 00] has a discussion of time-sharing and covers the technologies and the personalities involved in the development of the modern personal workstation.

The economics of the cloud are described in [Harms 10].

The Computer Measurement Group (CMG) is a not-for-profit worldwide organization that provides measurement and forecasting of various quantitative aspects of computer usage. Their measurements of the overhead due to virtualization can be found at www.cmg.org/measureit/issues/mit39/m_39_1.html.

If you want to learn more about TCP/IP and NAT, you can find a discussion at www.ipcortex.co.uk/wp/fw.rhtm.

The BigTable system is described in [Chang 06].

Netflix maintains a tech blog that is almost entirely focused on cloud issues. It can be found at techblog.netflix.com.

The home page for MongoDB is www.mongodb.org/display/DOCS/Home and for HBase is hbase.apache.org.

Michael Stonebraker is a database expert who has written extensively comparing NoSQL systems with RDBMSs. Some of his writings are [Stonebraker 09], [Stonebraker 10a], [Stonebraker 11], and [Stonebraker 10b].

Eric Brewer has provided a nice overview of the issues surrounding the CAP theorem for today's cloud-based systems: [Brewer 12].

26.9 Discussion Questions

1. "Service-oriented or cloud-based systems cannot meet hard-real-time requirements because it's impossible to guarantee how long a service will take to complete." Do you think this statement is true or false? In either case, identify the one or two categories of design decisions that are most responsible for the correctness (or incorrectness) of the statement.
2. "Using the cloud assumes your application is service oriented." Do you think this is true or false? Find some examples that would support that statement and, if it is not universally true, find some that would falsify it.
3. Netflix discussed their movement from Oracle to SimpleDB on their tech blog. They also discussed moving from SimpleDB to Cassandra. Describe their rationale for these two moves.
4. Netflix also describes their Simian Army in their tech blog. Which elements of the Simian Army could be offered as a SaaS? What would the design of such a SaaS look like?
5. Develop the "Hello World" application on an IaaS and on a PaaS.

This page intentionally left blank